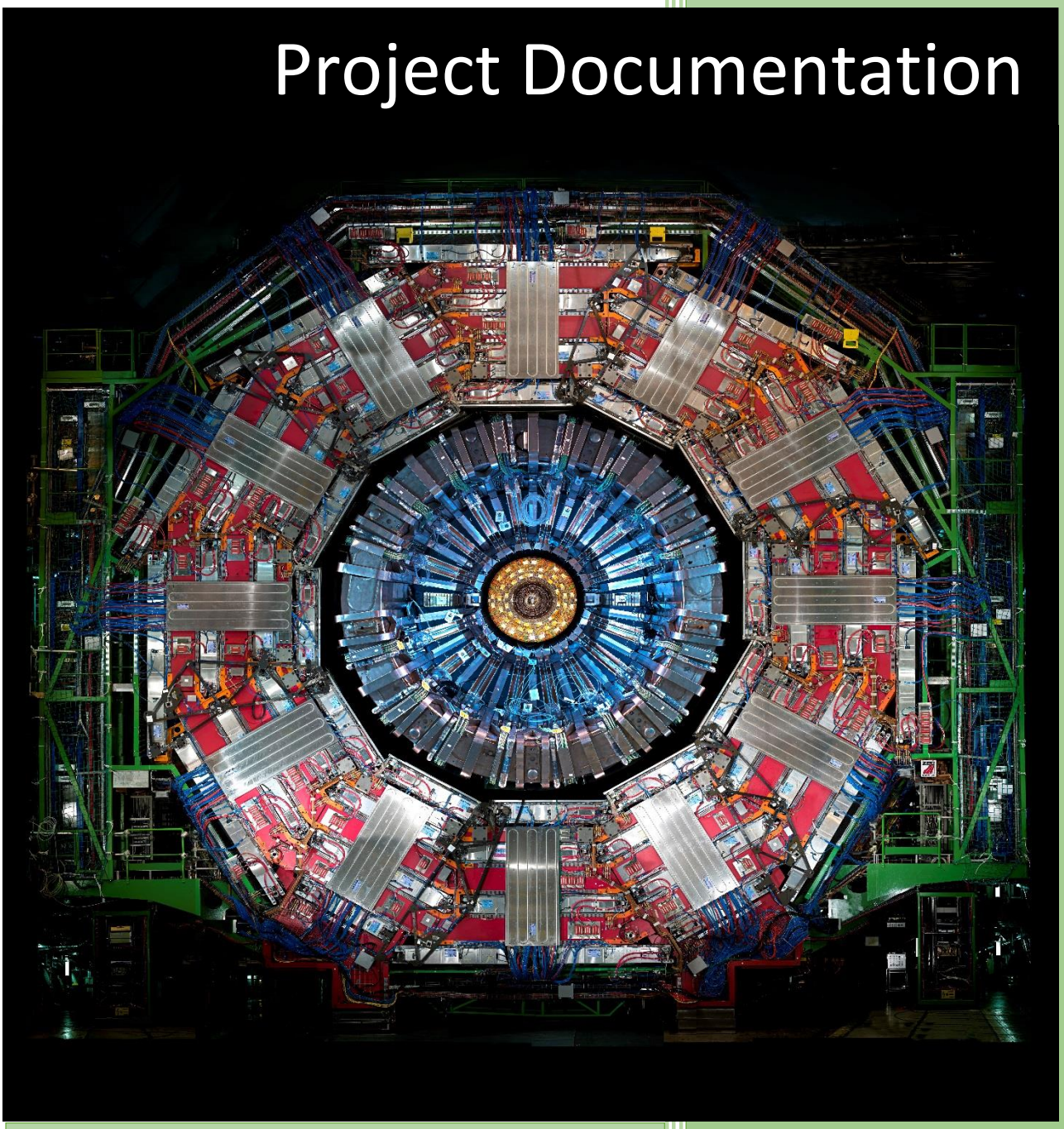
2016

Project Documentation



Configuration of SoC FPGA, Booting of HPS and running Bare Metal Application from SD card

Supervisors: Dominique GIGI

Petr Zejdl

Awais Zahid Rasheed

CERN Summer Student

9/6/2016

Aim:

The project goal is to evaluate the possible approaches for implementing a custom embedded system based on ALTERA SoC FPGA. The project requirements comprises the full automatic boot from SD card, configuration of the hardware and start of the software inside the embedded CPU.

Abstract:

First, a hardware design is created using Qsys in Quartus 16.0. Creation of the hardware design consists of configuring Hard Processor System (HPS) inside FPGA and adding necessary hardware blocks to the design. After generating the Qsys design, it is then instantiated in top level module in Verilog or VHDL. After setting up all pin assignments and adding all necessary files in the design, project is compiled to have a complete hardware design.

Second part comprises full software design in correspondence with the hardware design and booting the HPS from SD card. Software includes enabling the different bridges used by HPS to communicate with FPGA, configuring FPGA from HPS and embedded application itself. Finally, everything is added in the SD card to get a complete automatic bare metal application running on the host board without any configuration what so ever.

Hardware Used:

For this project, **TERASIC ALTERA CYCLONE V 5CSEMA5F31C6N** has been used. Features of this board are listed below:

FPGA:

- Altera Cyclone® V SE 5CSEMA5F31C6N device
- Altera serial configuration device – EPCS128
- USB-Blaster II on board for programming; JTAG Mode
- 64MB SDRAM (16-bit data bus)
- 4 push-buttons
- 10 slide switches
- 10 red user LEDs
- Six 7-segment displays
- Four 50MHz clock sources from the clock generator
- 24-bit CD-quality audio CODEC with line-in, line-out, and microphone-in jacks
- VGA DAC (8-bit high-speed triple DACs) with VGA-out connector
- TV decoder (NTSC/PAL/SECAM) and TV-in connector
- PS/2 mouse/keyboard connector
- IR receiver and IR emitter
- wo 40-pin expansion header with diode protection
- A/D converter, 4-pin SPI interface with FPGA

HPS:

- 800MHz Dual-core ARM Cortex-A9 MP Core processor
- 1GB DDR3 SDRAM (32-bit data bus)
- 1 Gigabit Ethernet PHY with RJ45 connector
- 2-port USB Host, normal Type-A USB connector
- Micro SD card socket

- Accelerometer (I2C interface + interrupt)
- UART to USB, USB Mini-B connector
- Warm reset button and cold reset button
- One user button and one user LED
- LTC 2x7 expansion header

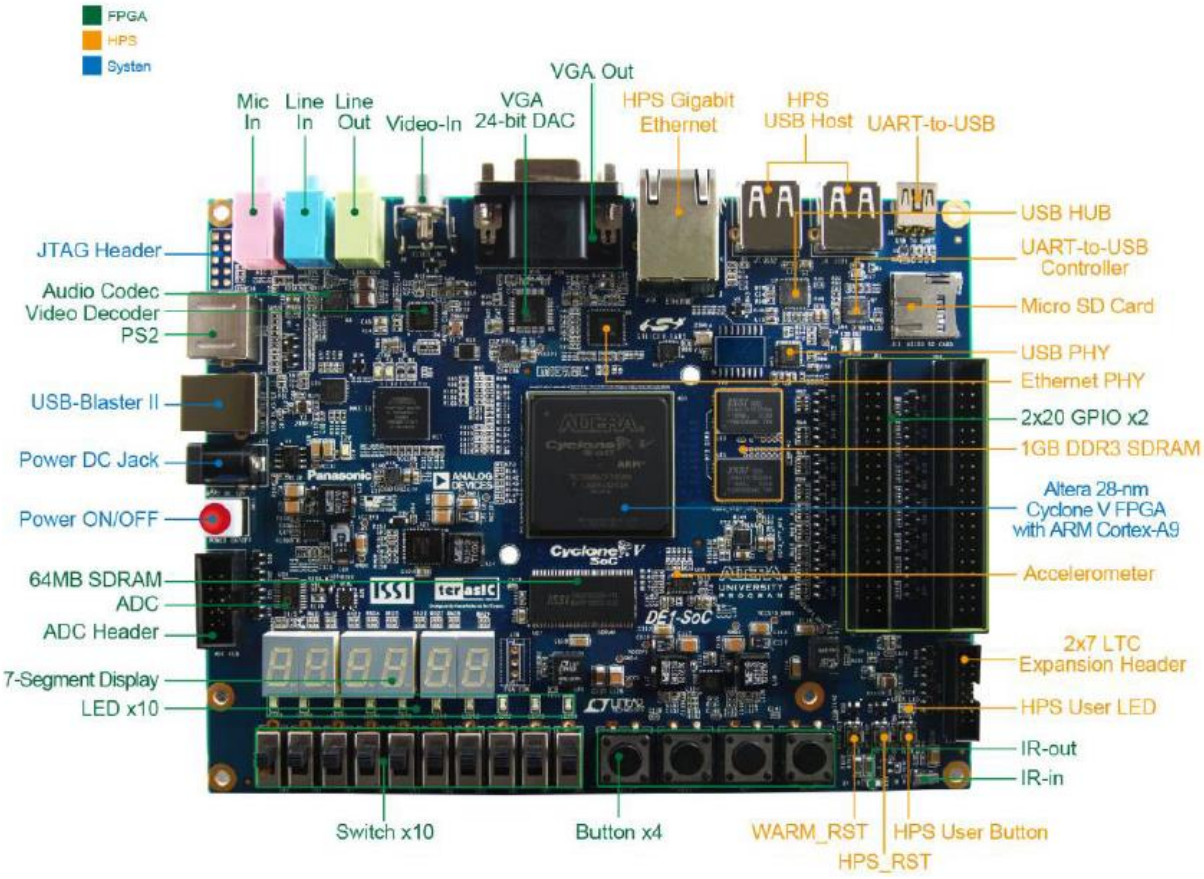


Figure 1: ALTERA Cyclone V FPGA-HPS

For more information about the board, go to this link:

https://www.altera.com/content/dam/altera-www/global/en_US/pdfs/literature/manual/rm_cv_soc_dev_board.pdf

Hard Processor System in Cyclone V

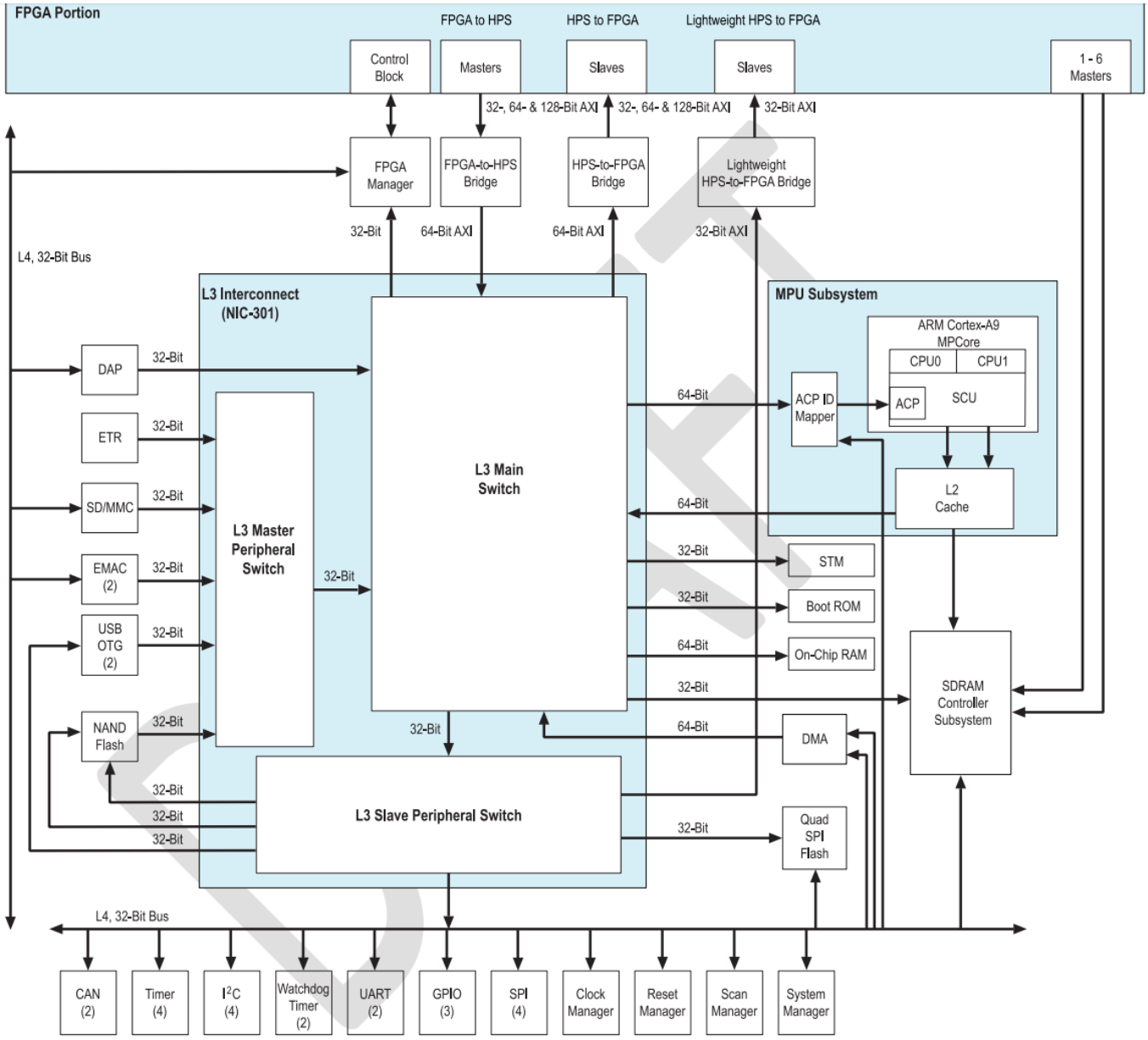


Figure 2: HPS Interconnections inside Cyclone V

Block diagram shows the three switches inside HPS but the most important one is L3 main switch as is controls the other switches and also the bridges. There are three bridges for different purposes. FPGA-HPS Bridge is used to control HPS from FPGA and vice versa. There are two types of HPS-FPGA bridges: One is **HPS-FPGA** and other one is **HPS-FPGA LW** (Light Weight). LW Bridge is used when control of FPGA peripherals is needed through FPGA while other one is used when FPGA is configured from HPS software. HPS-FPGA Bridge can be of 128 bits depending upon the specific task.

Now to access these bridges, **address mapping of these are very important to know**. Base addresses are given below:

- HPS-FPGA LW (0xFF20_0000)
- HPS-FPGA (0xC000_000)

These base addresses are fixed inside HPS. How to use these addresses will be explained later when we finish our HPS design in Qsys.

Building Hard Processor System (HPS) in QSYS:

It consists of following steps:

1. In Qsys library, go to **Processors and Peripherals < Hard Processor Systems** and select Arria V/Cyclone V Hard Processor System.
2. Now add **jtag to Avalon Master Bridge** from **Basics Functions > Bridges & Adapters > Memory Mapped** and name it (it is named system_console in figure 4). Also include on chip RAM for the design.
3. After that to see HPS booting sequence on terminal, we need to add **jtag URAT** into our design too. **Go to Interfaces & Protocols > Serial** and select jtag UART (named jtag_uart_0 in figure 6).
4. Add peripherals for specific application e.g. led or hex.
5. Connect all clocks and resets. Now for peripheral slaves, connect these with **h2f_lw master bridge** of HPS block as shown in figure 3.

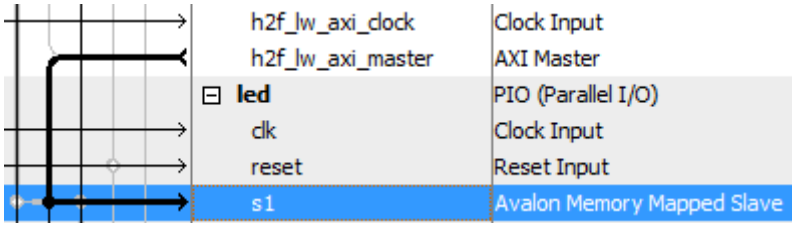


Figure 3: led slave

6. Now connect Master of jtag-Avalon with **f2h_axi_slave** as shown in figure 4:

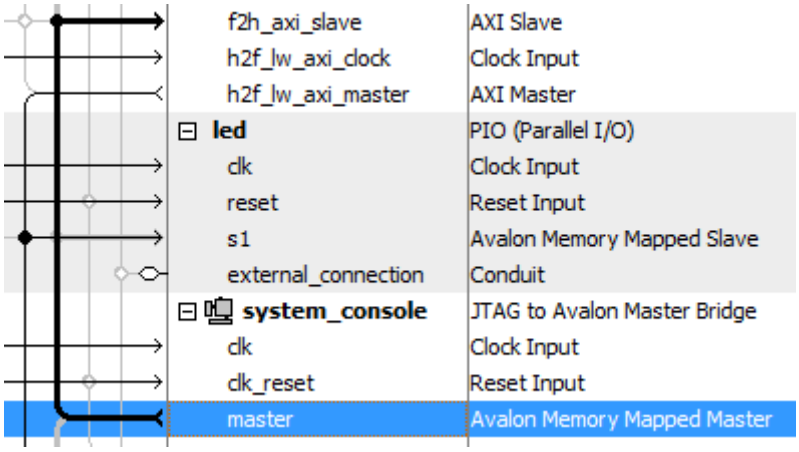


Figure 4: jtag-Avalon master

7. For On-Chip RAM, connect the slave s1 with f2h_axi_master as shown in figure 5:

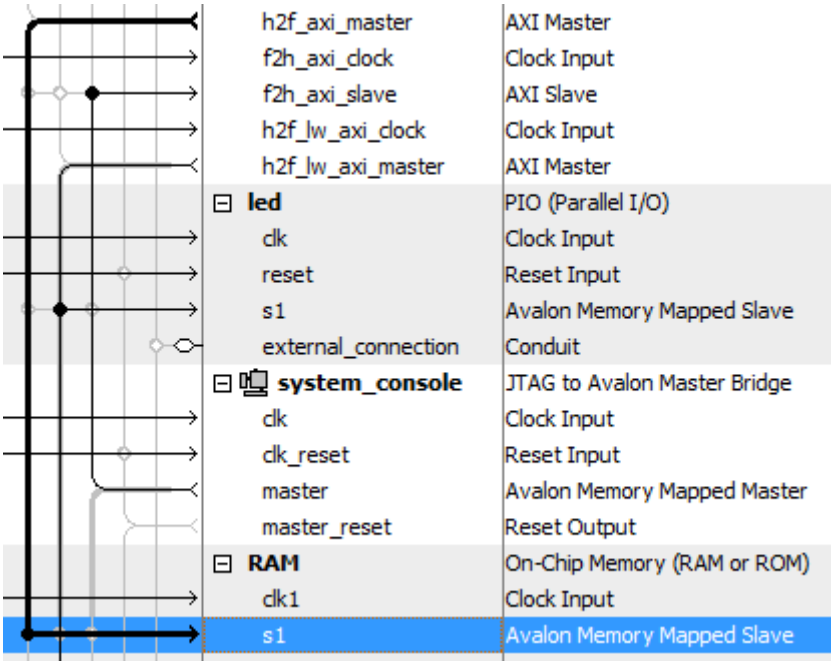


Figure 5: On chip Memory slave

8. Connect the slave of jtag UART with **h2f_lw_axi master** as shown in figure 6:

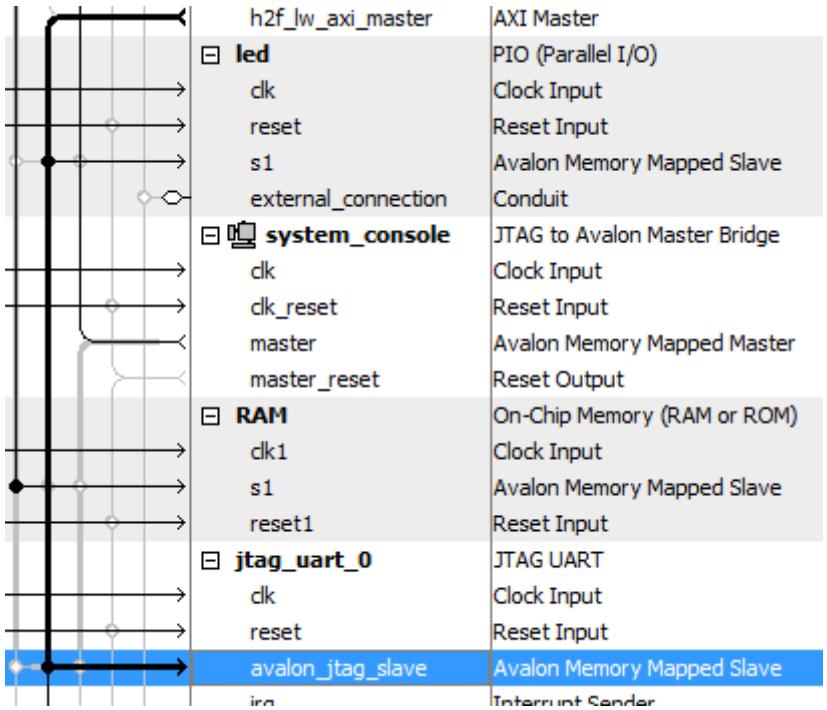
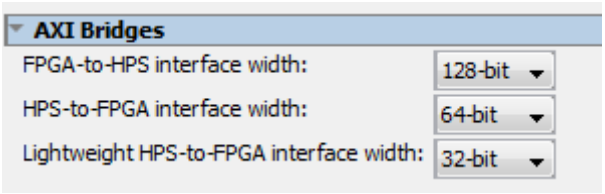


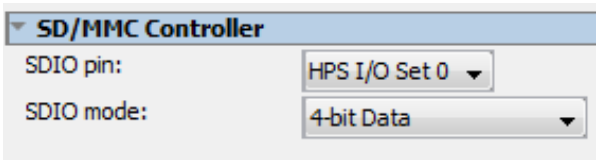
Figure 6: jtag-UART slave connection with HPS

9. Now that all connections have been made, it's time to set the HPS itself! Open the HPS and on FPGA interfaces tab, do the following:
    a. Change the number of bits of the bridges to the following:

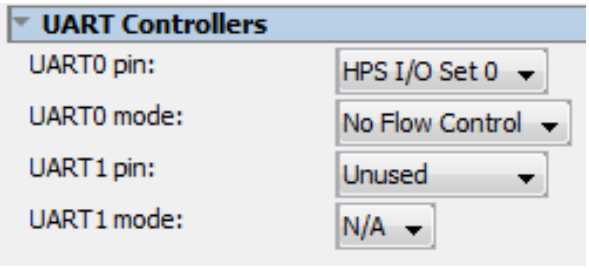


b. On HPS-FPGA option, tick the FPGA Manager Interrupt.

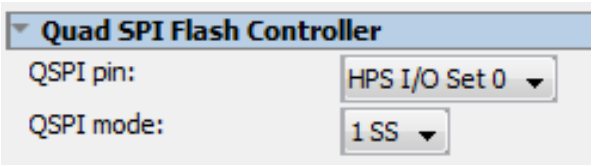
10. Now move to the other tab which is Peripheral Pins and do the following:
    c. Use the SDIO pin in SD/MMC Controller option:



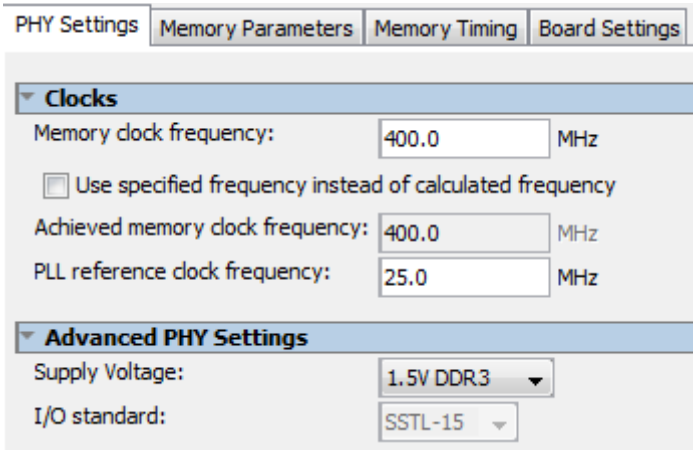
    d. Enable the UART0 pin in UART Controller for terminal use:



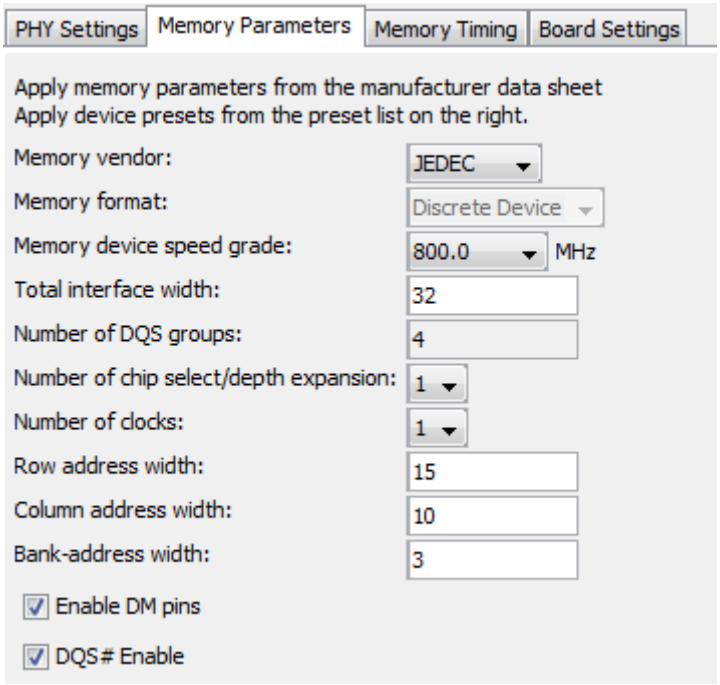
    e. If using QSPI too, enable it as well:



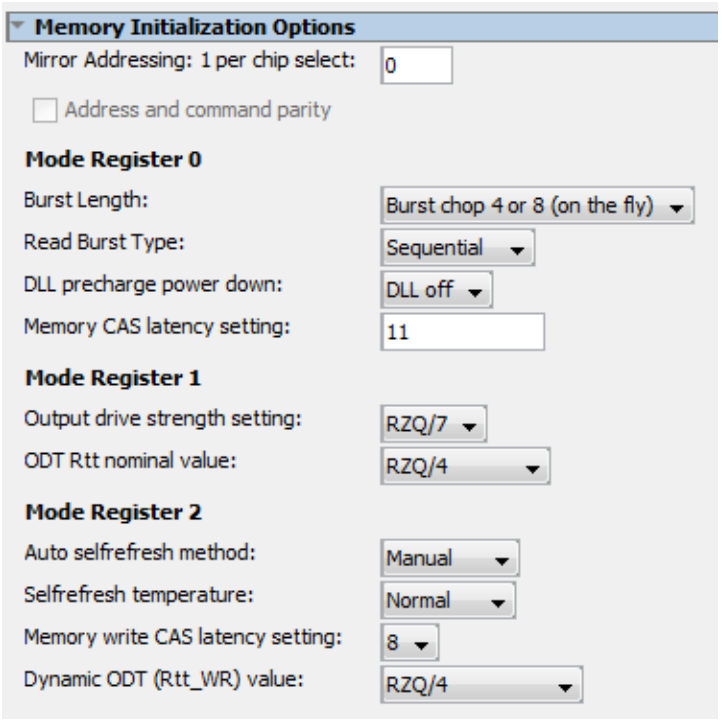
11. Now as peripherals are set, go to the HPS clocks tab and set the external clock sources to 25 MHz.
12. After that comes the settings of SDRAM. SDRAM protocol is by default set to DDR3 as HPS is connected with it internally. Now do the following:
    f. In the PHY settings tab:



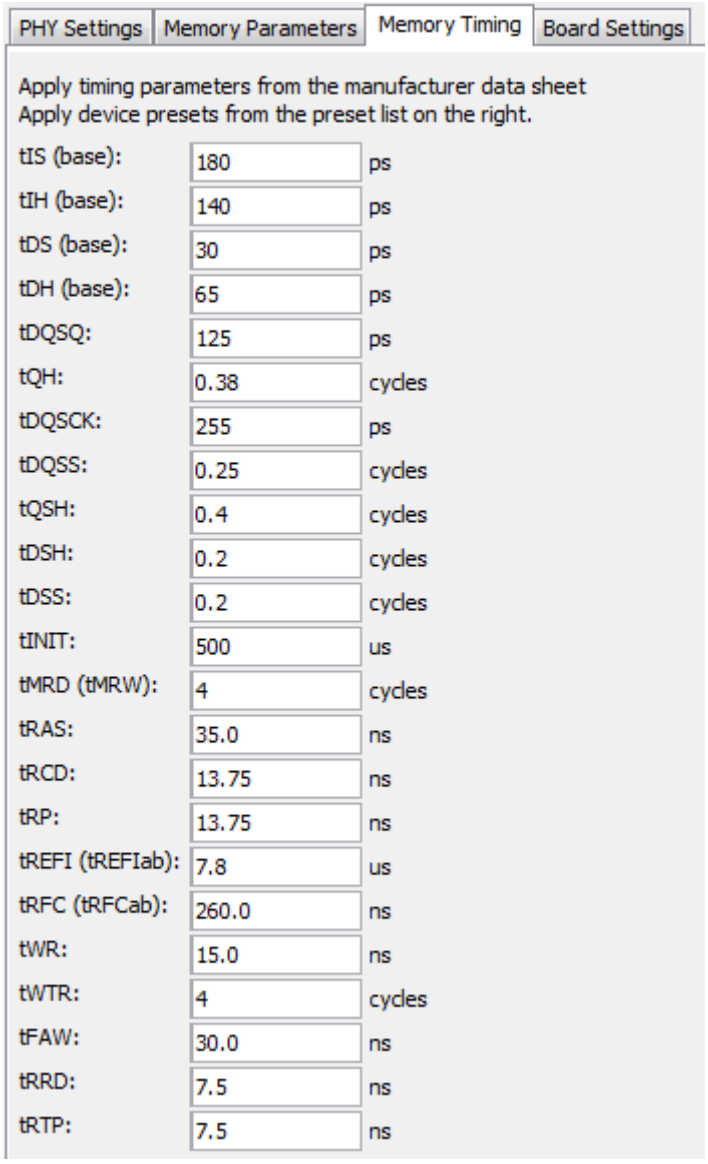
g. In memory parameters tab, lot of parameters have to be set in accordance with the board that has been used. As every board has different configurations for DDR memory. Before Memory initialization option, do the following:



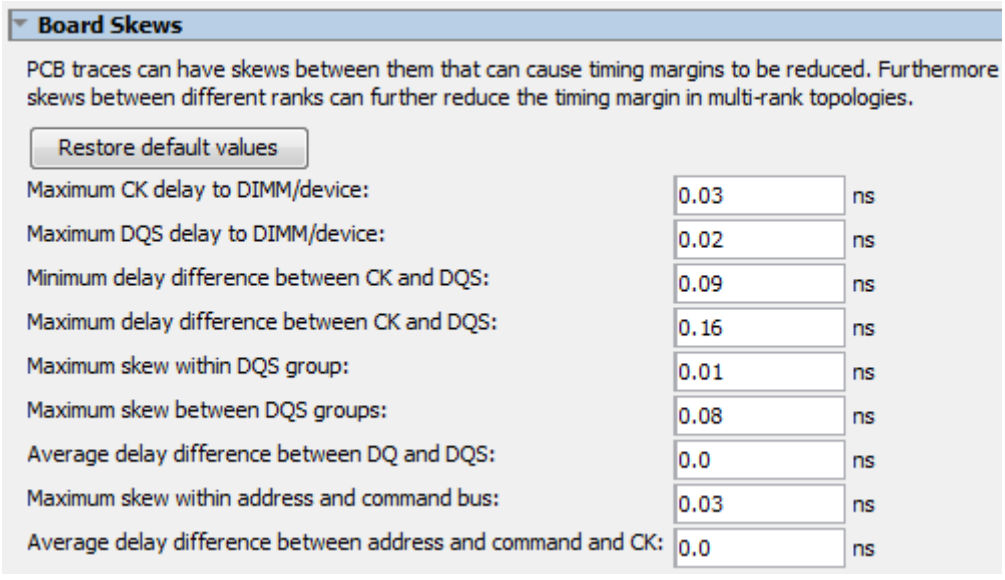
Now in memory initialization:



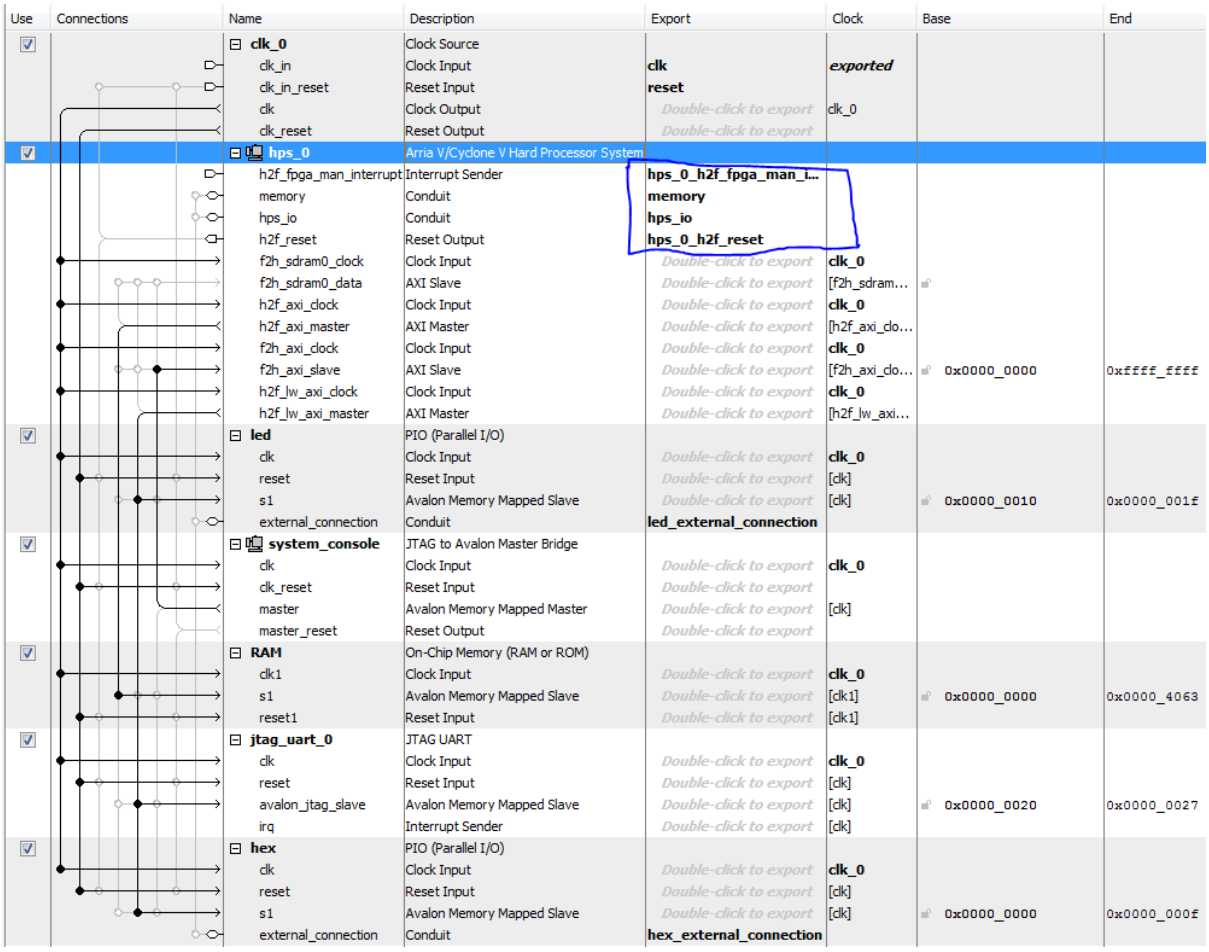
h. Now in Memory Timings tab:



i. Now in Board Settings tab, leave everything unchanged except for Board Skews and modify them according to figure:



Now that everything is build, time to export the conduits of individual blocks. For HPS, we need to export four connections as highlighted in figure 7 by double clicking the connection:



Use	Connections	Name	Description	Export	Clock	Base	End
☑		**clk_0**	Clock Source				
		clk_in	Clock Input	**clk**	***exported***		
		clk_in_reset	Reset Input	**reset**			
		clk	Clock Output	*Double-click to export*	clk_0		
		clk_reset	Reset Output	*Double-click to export*			
☑		**hps_0**	Arria V/Cyclone V Hard Processor System				
		h2f_fpga_man_interrupt	Interrupt Sender	**hps_0_h2f_fpga_man_i...**			
		memory	Conduit	**memory**			
		hps_io	Conduit	**hps_io**			
		h2f_reset	Reset Output	**hps_0_h2f_reset**			
		f2h_sdram0_clock	Clock Input	*Double-click to export*	**clk_0**		
		f2h_sdram0_data	AXI Slave	*Double-click to export*	[f2h_sdram...		
		h2f_axi_clock	Clock Input	*Double-click to export*	**clk_0**		
		h2f_axi_master	AXI Master	*Double-click to export*	[h2f_axi_clo...		
		f2h_axi_clock	Clock Input	*Double-click to export*	**clk_0**		
		f2h_axi_slave	AXI Slave	*Double-click to export*	[f2h_axi_clo...	0x0000_0000	0xffff_ffff
		h2f_lw_axi_clock	Clock Input	*Double-click to export*	**clk_0**		
		h2f_lw_axi_master	AXI Master	*Double-click to export*	[h2f_lw_axi...		
☑		**led**	PIO (Parallel I/O)				
		clk	Clock Input	*Double-click to export*	**clk_0**		
		reset	Reset Input	*Double-click to export*	[clk]		
		s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0000_0010	0x0000_001f
		external_connection	Conduit	**led_external_connection**			
☑		**system_console**	JTAG to Avalon Master Bridge				
		clk	Clock Input	*Double-click to export*	**clk_0**		
		clk_reset	Reset Input	*Double-click to export*			
		master	Avalon Memory Mapped Master	*Double-click to export*	[clk]		
		master_reset	Reset Output	*Double-click to export*			
☑		**RAM**	On-Chip Memory (RAM or ROM)				
		clk1	Clock Input	*Double-click to export*	**clk_0**		
		s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk1]	0x0000_0000	0x0000_4063
		reset1	Reset Input	*Double-click to export*	[clk1]		
☑		**jtag_uart_0**	JTAG UART				
		clk	Clock Input	*Double-click to export*	**clk_0**		
		reset	Reset Input	*Double-click to export*	[clk]		
		avalon_jtag_slave	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0000_0020	0x0000_0027
		irq	Interrupt Sender	*Double-click to export*	[clk]		
☑		**hex**	PIO (Parallel I/O)				
		clk	Clock Input	*Double-click to export*	**clk_0**		
		reset	Reset Input	*Double-click to export*	[clk]		
		s1	Avalon Memory Mapped Slave	*Double-click to export*	[clk]	0x0000_0000	0x0000_000f
		external_connection	Conduit	**hex_external_connection**			

Figure 7: All connections, Base addresses and exported conduits

As we can see, **led** has been assigned a base address of **0x0000_0010**, **hex** has been assigned **0x0000_0000** and **jtag_uart0** has been assigned **0x000_0020**. These can be changed (Double click on the address to edit it) and there is also a tool for assigning these addresses. To use that tool, go to **System > Assign Base Addresses .These addresses are very important for bare metal applications.** For further information about the configuration of HPS, please visit the link given below as it describes the **Golden Reference Design 16.0** for Cyclone V kit:

https://rocketboards.org/foswiki/view/Documentation/AVCVGSRD160

It is like an example SoC design for this kit and everything that is done above to configure HPS, used this reference design. Now save the design and generate the synthesis files for full design of Quartus.

For further information about Qsys design in Quartus, similar example design can be followed using **NIOS II processor** which is documented in the following link:

http://cmd64-cctrl-01.cern.ch/~FEROL/Altera/Hello_World_Lab_Manual.pdf

Quartus HDL design with Qsys:

When generating synthesis files, there is also an option for generating instantiation for hardware design. Include it in the main or top level module by calling it and also define the inputs and outputs for the corresponding variables. For further information about the input and output ports, go to the Appendix A. Now this design has two resets: One is a physical reset which can be set to any witch or key on the board but other one is HPS's h2f reset which is used by HPS itself. It can also be seen in figure 6 by name **hps_0_h2f_reset**.

After this create Synopsys Design Constraint file to create a clock for project (see Appendix B). Set the Verilog/VHDL design to top level entity. Now add all pin assignments along with SDRAM DDR3 and compile the project. If you get errors at place & route stage, it means that the design have not captured SDRAM parameters. In that case, go to **Tools > Tcl Scripts > submodules** and run **hps_sdram_p0_parameters.tcl.** After that run **hps_sdram_p0_assignments.tcl.** Now everything will be OK and programming file (SOF) is ready.

ARM software development with DS-5:

DS-5 provides the resources to debug your C application. In this case, no BIN file is needed as it creates its own debug file (.axf) and runs it. But for SD card, BIN file is needed to run the application. So two things are important in creating automatic bare metal application: one is to use hardware libraries (to set the bridges and so on) and the other is to create a BIN file. For this two projects are very important which are provided by Altera:

1. **Altera-SoCFPGA-HardwareLib-FPGA-CV-GNU**
2. **Altera-SoCFPGA-HardwareLib-Unhosted-CV-GNU**

1) Altera provides lot of design examples for bare metal and Linux applications**.** One of the project is "**Altera-SoCFPGA-HardwareLib-FPGA-CV-GNU**" which is very useful as it contains all hardware libraries that are necessary for program application. It consists of files which can set up HPS bridges, clocks and FPGA manager. For a simple bare metal application, only bridge manager can be used to control FPGA peripherals. So, in main C file, comment every function except for "s**ocfpga_bridge_setup ()**". Now in main function, four constants have been defined: ALT_LWH2F_SYSTEM_ID, ALT_LWH2F_BASE and ALT_LWH2F_LED_OFFSET. **Check that whether ALT_LWH2F_LED_OFFSET has the right value of address. It has to be the same base address as it was defined in Qsys**. Now comment every function calling except for **socfpga_bridge_setup ()** and write some code to access leds by using function as:

   ```
   alt_write_word(ALT_LWH2F_BASE + ALT_LWH2F_LED_OFFSET, value)
   ```

   The "value" could be an integer. After that build and debug the application. Check the console weather bridge setting is successful or not and leds are glowing according to program.

2) **Altera-SoCFPGA-HardwareLib-Unhosted-CV-GNU** is provided with a different Makefile to produce the required "BIN file". But this project does not have the luxuries of hardware libraries. So this project has to be used in combination with "**Altera-SoCFPGA-HardwareLib-FPGA-CV-GNU**" to get the required outcome.

Modification of "Altera-SoCFPGA-HardwareLib-Unhosted-CV-GNU":

To modify this project, follow these steps:

- Add all C files that were there in the first project:

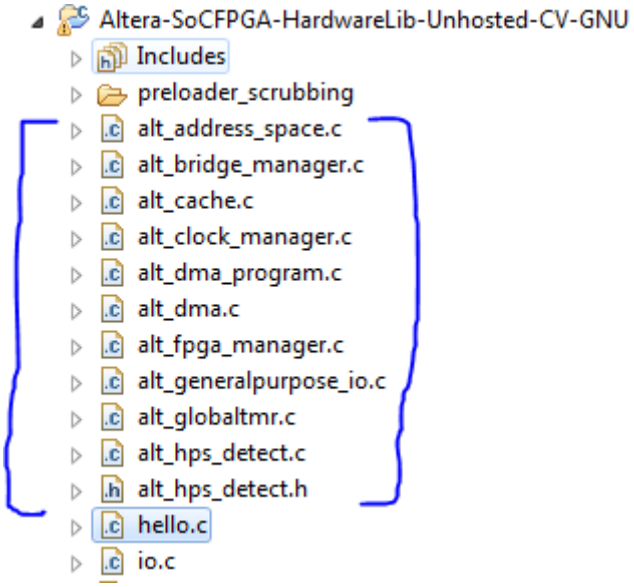


Figure 8: All necessary C files

- Now open the Makefile and add all these files in it:

```
5 ALT_DEVICE_FAMILY ?= soc_cv_av
6 SOCEDS_ROOT ?= $(SOCEDS_DEST_ROOT)
7 HWLIBS_ROOT = $(SOCEDS_ROOT)/ip/altera/hps/altera_hps/hwlib
8
9 HWLIBS_SRC  :=
10 EXAMPLE_SRC := hello.c io.c alt_bridge_manager.c alt_cache.c alt_clock_manager.c
11 alt_fpga_manager.c alt_address_space.c alt_generalpurpose_io.c alt_hps_detect.c
12
13 C_SRC       := $(EXAMPLE_SRC) $(HWLIBS_SRC)
14
15 LINKER_SCRIPT := cycloneV-dk-ram-modified.ld
```

Figure 9: Inclusion of all C files in Makefile

- Add all Hardware libraries which were included in the first project:

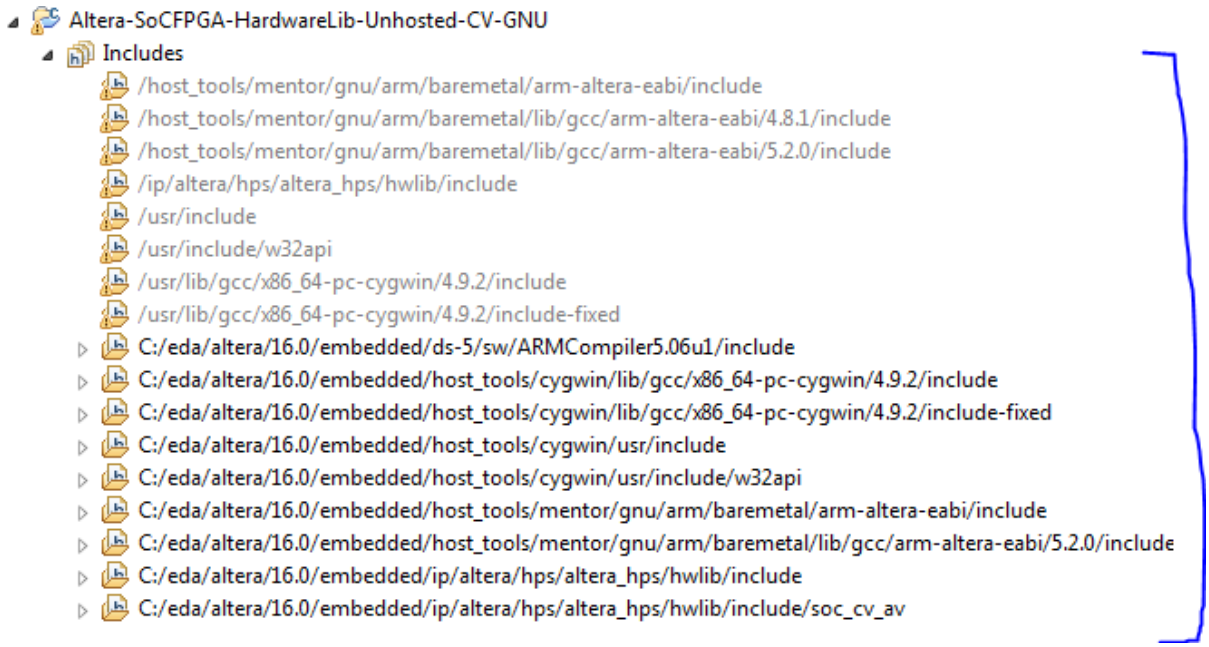


Figure 10: All necessary Hardware libraries

- Now change the content of "**hello.c**" according to "**hwlib.c**" in the first project to perform bridge setup etc. and build the project.
- After building, you will see the required BIN file:

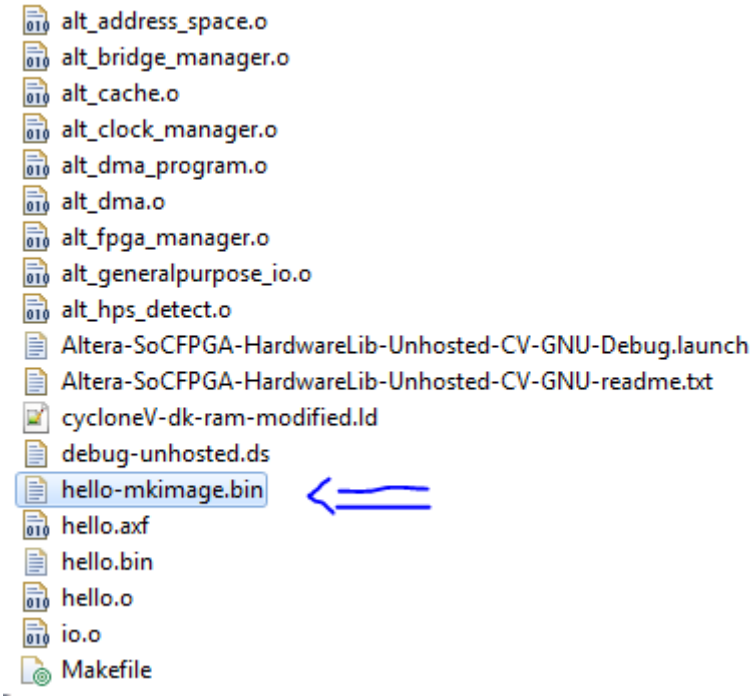


Figure 11: Object, debug and BIN files

Now application file is ready for SD card!

Booting HPS from SD/MMC card:

To boot HPS from SD card, certain things are required:

I. Preloader
II. Bootloader
III. Device tree file
IV. Linux image

Last two things are not required when Linux is not used. Now to run a bare metal application with a SD card, there are two ways: From a Preloader only or Preloader and Bootloader both.

a. Bare Metal Application with a Preloader:

Running the application from preloader does not include programming the FPGA. It has to be programed before with some programming tool to run the application. It consists of following steps:

I. Open up **Embedded Command Shell 16.0** and type:
   $ **bsp-editor**

II. A window will appear up. Go to **File > New HPS BSP** and choose the directory where hardware-software-handshake files (**hps_isw_handoff**) are situated which were made by Quartus.
III. By default, Preloader loads the next image from **0x40_000** and it is always loading the **uboot.img.** So to load an application image, one need to replace the **uboot.img** with **<application_name>.bin** as described in the figure 12:
IV. Next go to **Advanced < spl < boot** and uncheck the **WATCHDOG_ENABLE** but check both **SDRAM_SCRUBBING** and **SDRAM_SCRUB_REMAIN_REGION** as shown in figure 13**.** Now generate!
V. In embedded command shell, go to the directory, you chose for preloader e.g.
   $ **cd d:/project_student/Full_final_Project/software/spl_bsp**

   And type:
   $ **make**

   This will make Preloader image naming: "**preloader-mkimage.bin**".
VI. Now time to write this preloader image into SD card. To do that type:
   $ **alt-boot-disk-util.exe –p preloader-mkimage.bin –a write –d e**

   Where **e** is the drive letter.
VII. Now copy your application BIN file into SD card and insert it into board.

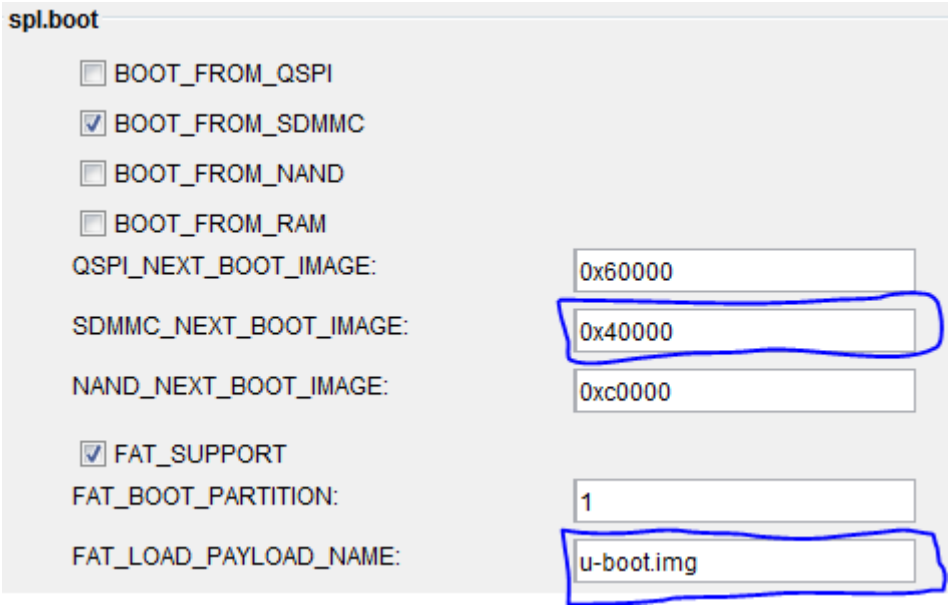


Figure 12: Preloader Configuration in bsp-editor

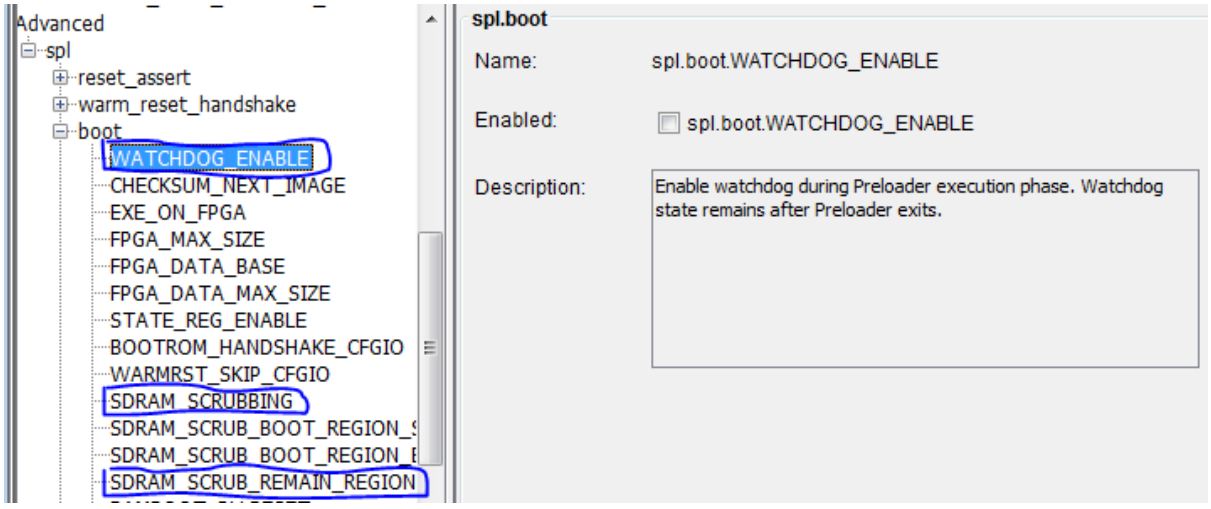


Figure 13: Further Customization

b. Bare Metal Application from Bootloader:

After preloader booting is complete, it goes on to run the next image stored on the address **0x40000**. In this case, there has to be a bootloader image file on that address because next booting stage is a bootloader. With this, it is possible to configure FPGA and run the application both. So to do this, three additional things are needed which are boot image, boot script and RBF file (programming file).

- **Generation of RBF file from SOF file:**

Open **Nios-II command shell 16.0** (Note that it is not the same as **Embedded Command Shell 16.0**) and go to the directory where SOF file is located. Now type following commands:

$ **sof2flash --offset=0 --input="./your_file.sof" --output="./your_file.flash"**

This will create a flash file but we need a RBF file. So type:

$ **nios2-elf-objcopy –I srec –O binary "./your_file.flash" "./your_file.rbf"**

Now RBF file is ready. Copy it into SD card!

➢ **Creating bootloader image:**

Before creating this image, preloader has to be set to load it. Check that whether **FAT_LOAD_PAYLOAD_NAME** is set like this in the **bsp-editor** console:

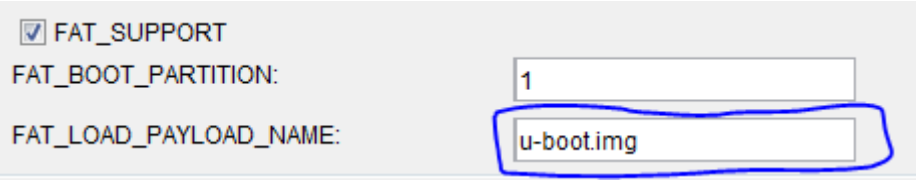


Figure 14: Boot image name

Now go to the preloader directory and type (in embedded command shell 16.0):

$ **make uboot**

It will create u-boot.img. Now to write it on SD card, type:

$ **cp uboot-socfpga/u-boot.img /cygdrive/e**

➢ **Creating Boot Script for Boot loader:**

When bootloader is loaded, it runs some commands which is written down on boot script. So to create boot script, do the following:

- Open a text file and type following commands in it:

```
echo ---Loading FPGA firmware---;
fatload mmc 0:1 $fpgadata Your_RBF_file_Name.rbf;
fpga load 0 $fpgadata $filesize;
echo ---Done---;
;
#echo ---Enabling bridge---;
#run bridge_enable_handoff;
#echo ---Done---;
;
echo ---Loading application image into memory---;
fatload mmc 0 0x100000 Your_Application_file_name.bin;
echo ---Done---;
;
echo ---Starting application from memory---;
go 0x100040;
```

The commands starting from “echo”, are just from printing information on the terminal. The second line loads the RBF programming file from SD card into the memory of FPGA and then next command configures the FPGA from it. After that bridge is enabled but in this example it has been commented because we are enabling the bridge from software i.e. by application (BIN file) file itself. So there is no need for this command in this boot script. Next step is to load the application file (BIN file). Now this is tricky as this file has to be loaded from specific address which depends upon linker script. To check what linker script is used by the make file, go to the project directory and open make file in notepad++. Check the name of the **linker script**:

```
ALT_DEVICE_FAMILY ?= soc_cv_av
SOCEDS_ROOT ?= $(SOCEDS_DEST_ROOT)
HWLIBS_ROOT = $(SOCEDS_ROOT)/ip/altera/hps/altera_hps/hwlib

HWLIBS_SRC  :=
EXAMPLE_SRC := hello.c io.c alt_bridge_manager.c alt_cache.c al
C_SRC       := $(EXAMPLE_SRC) $(HWLIBS_SRC)

LINKER_SCRIPT := cycloneV-dk-ram-modified.ld

MULTILIBFLAGS := -mcpu=cortex-a9 -mfloat-abi=softfp -mfpu=neon
```

Figure 15: Linker Script name in Makefile

Now open this linker script and find the memory address to where it will load the application:

```
OUTPUT_FORMAT ("elf32-littlearm", "elf32-bigarm", "elf32-littlearm")
ENTRY(__cs3_reset)
SEARCH_DIR(.)
GROUP(-lgcc -lc -lcs3 -lcs3unhosted -lcs3arm)

MEMORY
{
  boot_rom (rx) : ORIGIN = 0xfffd0000, LENGTH = 64K
  oc_ram (rwx) : ORIGIN = 0xffff0000, LENGTH = 64K
  /* Need to have 64bytes available before start of program, to store the mkimage header */
  ram (rwx) : ORIGIN = 0x100000 + 0x40, LENGTH = 1023M - 0x40
}

/* These force the linker to search for particular symbols from
 * the start of the link process and thus ensure the user's
 * overrides are picked up
 */
```

Figure 16: Address information to load application

As here it can be seen that mkimage header is stored at **0x100000** and it needs 64 bytes, hence using all addresses between **0x100000 – 0x100040.** It means that application file has to be loaded at **0x100000** but it should run from **0x100040.**

- After that save it by the name “u-boot.txt” or by any other name. Save this file by going into bootloader directory i.e. **software < spl_bsp < uboot-socfpga < tools**
- Next step is to change this text file into script (.scr) file. First, in the command shell, go to same directory where you saved this text file and type following command in embedded command shell:

$ **mkimage -A arm -O linux -T script -C none -a 0 -e 0 -n "My script" -d u-boot.text u-boot.scr**

This will create u-boot.scr (screen saver file). Copy this file into SD card.

- Now copy the application file (BIN file) into SD card too.

In the end our SD card should look like this:

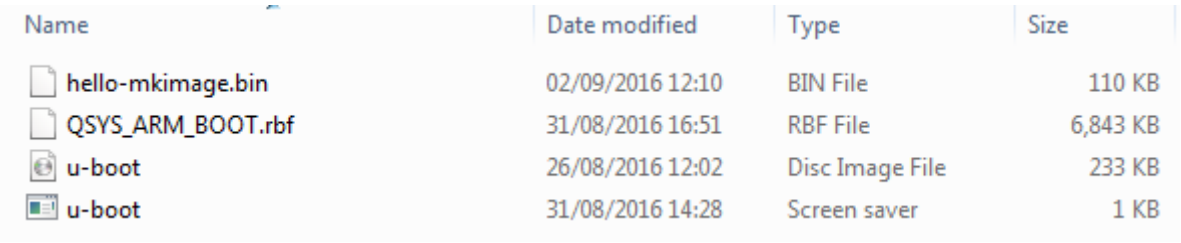


Figure 17: All contents on SD card

There are only four things showing but fifth thing is the preloader image which is stored in the FAT partition of the SD card and it is hidden. Now insert the SD card in the board and see the results on the terminal.

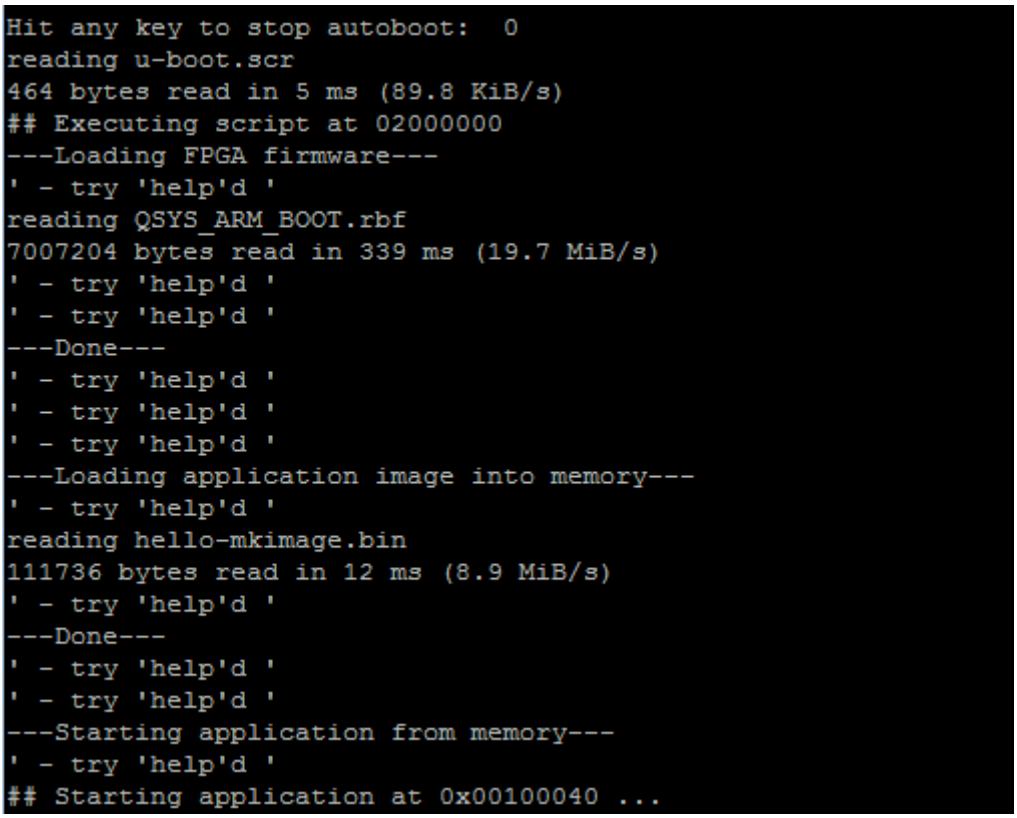


Figure 18: Command flow step by step on putty terminal

Appendix A: Top Level Verilog Module

```
module QSYS_ARM_BOOT_top (

 input clk_clk,reset,

output     [14:0] HPS_DDR3_ADDR,
output     [2:0]  HPS_DDR3_BA,
output            HPS_DDR3_CAS_N,
output            HPS_DDR3_CKE,
output            HPS_DDR3_CK_N,
output            HPS_DDR3_CK_P,
output            HPS_DDR3_CS_N,
output     [3:0]  HPS_DDR3_DM,
inout      [31:0] HPS_DDR3_DQ,
inout      [3:0]  HPS_DDR3_DQS_N,
inout      [3:0]  HPS_DDR3_DQS_P,
output            HPS_DDR3_ODT,
output            HPS_DDR3_RAS_N,
output            HPS_DDR3_RESET_N,
input            HPS_DDR3_RZQ,
output            HPS_DDR3_WE_N,
inout      [3:0]  HPS_FLASH_DATA,
output            HPS_FLASH_DCLK,
output            HPS_FLASH_NCSO,
output            HPS_SD_CLK,
inout            HPS_SD_CMD,
inout      [3:0]  HPS_SD_DATA,
input            HPS_UART_RX,
output            HPS_UART_TX,
output     [31:0]  led_external_connection_export,
```

```
output      [6:0]  hex_external_connection_export,
output wire        hps_0_h2f_reset_reset_n
                );


QSYS_ARM_BOOT u0 (
      .clk_clk                       ( clk_clk ),
      .reset_reset_n                 ( reset),
      .memory_mem_a                  ( HPS_DDR3_ADDR),
      .memory_mem_ba                 ( HPS_DDR3_BA),
      .memory_mem_ck                 ( HPS_DDR3_CK_P),
      .memory_mem_ck_n               ( HPS_DDR3_CK_N),
      .memory_mem_cke                ( HPS_DDR3_CKE),
      .memory_mem_cs_n               ( HPS_DDR3_CS_N),
      .memory_mem_ras_n              ( HPS_DDR3_RAS_N),
      .memory_mem_cas_n              ( HPS_DDR3_CAS_N),
      .memory_mem_we_n               ( HPS_DDR3_WE_N),
      .memory_mem_reset_n            ( HPS_DDR3_RESET_N),
      .memory_mem_dq                 ( HPS_DDR3_DQ),
      .memory_mem_dqs                ( HPS_DDR3_DQS_P),
      .memory_mem_dqs_n              ( HPS_DDR3_DQS_N),
      .memory_mem_odt                ( HPS_DDR3_ODT),
      .memory_mem_dm                 ( HPS_DDR3_DM),
      .memory_oct_rzqin              ( HPS_DDR3_RZQ),

      .hps_io_hps_io_qspi_inst_IO0   ( HPS_FLASH_DATA[0]   ),
      .hps_io_hps_io_qspi_inst_IO1   ( HPS_FLASH_DATA[1]   ),
      .hps_io_hps_io_qspi_inst_IO2   ( HPS_FLASH_DATA[2]   ),
      .hps_io_hps_io_qspi_inst_IO3   ( HPS_FLASH_DATA[3]   ),
      .hps_io_hps_io_qspi_inst_SS0   ( HPS_FLASH_NCSO   ),
      .hps_io_hps_io_qspi_inst_CLK   ( HPS_FLASH_DCLK   ),
```

```
.hps_io_hps_io_sdio_inst_CMD  ( HPS_SD_CMD    ),
.hps_io_hps_io_sdio_inst_D0     ( HPS_SD_DATA[0]    ),
.hps_io_hps_io_sdio_inst_D1     ( HPS_SD_DATA[1]    ),
.hps_io_hps_io_sdio_inst_CLK    ( HPS_SD_CLK   ),
.hps_io_hps_io_sdio_inst_D2     ( HPS_SD_DATA[2]    ),
.hps_io_hps_io_sdio_inst_D3     ( HPS_SD_DATA[3]    ),
.hps_io_hps_io_uart0_inst_RX    ( HPS_UART_RX    ),
.hps_io_hps_io_uart0_inst_TX    ( HPS_UART_TX    ),
.hps_0_h2f_reset_reset_n         ( hps_0_h2f_reset_reset_n),


.led_external_connection_export  (led_external_connection_export),
.hex_external_connection_export  (hex_external_connection_export)
    );
Endmodule
```

Appendix B: Synopses Design Constraint File

```
create_clock -period 20 [get_ports clk_clk]

create_clock -name {altera_reserved_tck} -period 40 {altera_reserved_tck}
set_input_delay -clock altera_reserved_tck -clock_fall 3 [get_ports altera_reserved_tdi]
set_input_delay -clock altera_reserved_tck -clock_fall 3 [get_ports altera_reserved_tms]
set_output_delay -clock altera_reserved_tck 3 [get_ports altera_reserved_tdo]

set_false_path -from * -to [get_ports {led_export[*]}]

create_clock -period "1 MHz" [get_ports hps_io_hps_io_i2c0_inst_SCL]
create_clock -period "48 MHz" [get_ports hps_io_hps_io_usb1_inst_CLK]
```